# The Human Testament API Documentation

*A walkthrough of the implemented manuscript and account APIs*

| Project | Current reference |
| --- | --- |
| Local API base | http://localhost:5000/api (PORT defaults to 5000) |
| Stack | Node.js, Express 5, MongoDB Atlas, Mongoose 9; React/Vite frontend |
| Snapshot | 22 October 2026 - implemented account Tasks 1-7 |

Use this note like the supplied Timely reference: find a route, check its authentication requirements, then follow the request and response example. The route index includes all 25 currently registered API operations.

## Two independent starting points

- Read the manuscript without an account: list books, open a chapter, then resolve a fixed verse reference.
- Manage an account: register, verify email when delivery is configured, log in, view/edit your own profile, and log out.

## Headers used by account requests

### Account request headers

```
Authorization: Bearer <access-token>
Origin: http://localhost:5173
Content-Type: application/json
X-CSRF-Protection: 1
```

Bearer is required only where stated. Every account POST/PATCH also needs the configured exact Origin, JSON content type and CSRF header, even when the route does not require login. Logout uses its refresh cookie, not a submitted user ID.

*HTTP examples assume the explicit loopback-only development exception. Production requires HTTPS and a configured trusted frontend origin. Replace token/ID placeholders with values from your own test flow. Accounts, passwords, IDs and timestamps in examples are synthetic.*

# Route index

*All paths below are relative to the API base. Public means no account credential is required.*

## Public manuscript and health

| Method | Route | Success |
|---|---|---|
| GET | /health | 200 |
| GET | /books | 200 |
| GET | /books/:slug | 200 |
| GET | /books/:slug/chapters | 200 |
| GET | /books/:slug/chapters/:chapterNumber | 200 |
| GET | /books/:slug/chapters/:chapterNumber/verses | 200 |
| GET | /books/:slug/chapters/:chapterNumber/verses/:verseNumber | 200 |
| GET | /books/:slug/chapters/:chapterNumber/verses/:verseNumber/supporting-material | 200 |
| GET | /verses/:canonicalId | 200 |
| GET | /glossary | 200 |
| GET | /cross-references | 200 |
| GET | /cross-references/index | 200 |
| GET | /sections?kind=front_matter | 200 |
| GET | /search?q=freedom | 200 |

## Accounts and profiles

| Method | Route | Access / success |
|---|---|---|
| POST | /auth/register | Public + mutation headers; 201 |
| POST | /auth/login | Public + mutation headers; 200 |
| POST | /auth/logout | Cookie + mutation headers; 200 |
| GET | /auth/me | Bearer; 200 |
| POST | /auth/forgot-password | Public + mutation headers; 202 |
| POST | /auth/reset-password | Reset token + mutation headers; 200 |
| POST | /auth/change-password | Bearer + mutation headers; 200 |
| POST | /auth/verify-email | Verification token + headers; 200 |
| POST | /auth/resend-verification | Public + mutation headers; 202 |
| GET | /users/me | Bearer; 200 |
| PATCH | /users/me | Bearer + mutation headers; 200 |

# 1. Read books and chapters

*The default PDF-derived source can serve public content without account login.*

## Check that the API is reachable

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| GET | /health | Public; 200 |

### Illustrative response

```
{
  "success": true,
  "message": "The Human Testament API is running.",
  "timestamp": "2026-10-22T10:00:00.000Z"
}
```

Health reports that the HTTP application is running. It does not, by itself, prove Atlas connectivity, account readiness, email delivery or publication approval.

## Find a book and open a chapter

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| GET | /books; /books/freedom; /books/freedom/chapters/10 | Public; 200 |

GET /books returns an array of book metadata. Use slug, not a MongoDB account ID, to navigate. GET /books/:slug/chapters returns chapter metadata. GET /books/:slug/chapters/:chapterNumber returns the book, chapter, verses, notes, cross-references and supporting sections.

### Illustrative shape only - abbreviated book list; metadata omitted

```
{
  "success": true,
  "count": 10,
  "data": [
    {
      "slug": "inquiry",
      "title": "Inquiry",
      "order": 1,
      "chapterCount": 41
    }
  ]
}
```

This excerpt illustrates the envelope. Read the full book list and current metadata from the API.

## Reference and fidelity rules

The current source contains 10 books, 230 chapters, 3,905 reference slots and 3,874 authored verses. 31 references have no authored source text; they remain explicitly unresolved. There are 1,470 editorial notes.

Repeated wording remains at each printed verse address. Do not deduplicate by text or renumber references. A missing-source record has status "missing_source_text" and unresolved evidence instead of invented verse text. The PDF and backend retain the same reference system.

# 2. Resolve a verse and its context

*A canonical verse ID has the form book-slug:chapter:verse.*

## Read a fixed verse reference

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| GET | /verses/freedom:10:14 | Public; 200 |

### Actual current verse text - selected fields; full provenance/support omitted


```
{
  "success": true,
  "data": {
    "canonicalId": "freedom:10:14",
    "bookSlug": "freedom",
    "chapterNumber": 10,
    "number": 14,
    "status": "authored",
    "text": "Responsibility is the shadow cast by freedom.",
    "source": {
      "pdfPage": 85,
      "pdfPages": [
        85
      ]
    }
  }
}
```


The equivalent nested route is /books/freedom/chapters/10/verses/14. The chapter verse-list route returns an array. Full verse responses also carry notes, crossReferences and source provenance; the default PDF source includes sourceSha256.

## Supporting-material routes

| GET route | What it returns |
|---|---|
| /books/:slug/chapters/:chapterNumber/verses/:verseNumber/supporting-material | canonicalId, status, associated notes and crossReferences |
| /glossary | Complete glossary entries |
| /cross-references | Anchored editorial citations |
| /cross-references/index | The manuscript cross-reference index |
| /sections | Front matter, back matter and chapter supporting sections |

Optional section kinds are front_matter, back_matter and chapter_supporting_material. Unknown kinds return 400. Editorial notes are canonical supporting material; they are not private reader notes.

## Search

GET /search?q=responsibility returns matching verse records with kind, reference and verse. In default PDF mode, matching is case-insensitive over reference labels and verse text; blank queries return an empty array. The database provider currently lacks search support, so do not assume search parity between content modes.

# 3. Register and log in

*Account creation does not grant email ownership or automatically create a login session.*

## Register a reader account

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| POST | /auth/register | No Bearer; 201 |

### Request body

```
{
  "email": " Reader@Example.Test ",
  "password": "A synthetic passphrase!",
  "displayName": "Reader"
}
```

### Illustrative registration response

```
{
  "success": true,
  "data": {
    "user": {
      "id": "507f1f77bcf86cd799439011",
      "email": "reader@example.test",
      "displayName": "Reader",
      "role": "user",
      "emailVerified": false,
      "isActive": true
    }
  }
}
```

Email is trimmed/lowercased and uniquely indexed regardless of casing. Display names are trimmed, 1-80 characters, with no embedded controls. Only the three fields above are accepted. Duplicate email returns 409 and reveals account existence.

## Log in

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| POST | /auth/login | No Bearer; 200 |

### Request body

```
{
  "email": "reader@example.test",
  "password": "A synthetic passphrase!"
}
```

Success returns data.user, data.accessToken and data.expiresIn (900 seconds), and sets the HttpOnly refresh cookie. Unknown email, wrong password and inactive accounts share the same safe 401 message. Only the user role is currently defined.

Passwords: 12+ code points, max 72 UTF-8 bytes, nonblank; no ASCII controls. bcrypt 12; no trimming/truncation.

# 4. Understand the login session

*Access is signed and also checked against current account/session state.*

### Illustrative login response

```
{
  "success": true,
  "data": {
    "user": {
      "id": "507f1f77bcf86cd799439011",
      "email": "reader@example.test",
      "displayName": "Reader",
      "role": "user",
      "emailVerified": false,
      "isActive": true
    },
    "accessToken": "<access-token>",
    "expiresIn": 900
  }
}
```

## Use the two credentials correctly

- Send the access token as Authorization: Bearer <access-token> on protected requests. It lasts 15 minutes.

- The 30-day refresh credential is delivered only in a cookie. Production name: __Secure-htt_refresh; HttpOnly, Secure, SameSite=Lax, Path=/api/auth.

- Only a hash of the refresh credential is stored. Browser JavaScript does not read the cookie, and refresh tokens never appear in response JSON.

- The frontend auth service keeps access only in memory. No credential is written to localStorage/sessionStorage.

## Read the existing authentication profile

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| GET | /auth/me | Bearer; 200 |

Response: success is true and data.user contains the same six auth-safe fields shown in the registration example.

/auth/me returns the established auth-safe fields, including role/isActive. For account settings with the creation date and a narrower response, use /users/me on the next page. Neither endpoint takes a target user ID.

**There is no POST /auth/refresh, refresh rotation or automatic renewal endpoint yet. A page reload or access expiry therefore requires login again. The cookie lifetime is not a promise that renewal already works.**

# 5. View and edit account settings

*The authenticated user can edit only their own display name.*

## Retrieve the safe settings profile

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| GET | /users/me | Bearer; 200 |

### Illustrative response

```
{
  "success": true,
  "data": {
    "user": {
      "id": "507f1f77bcf86cd799439011",
      "displayName": "Reader",
      "email": "reader@example.test",
      "emailVerified": false,
      "createdAt": "2026-10-22T10:00:00.000Z"
    }
  }
}
```

## Update the display name

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| PATCH | /users/me | Bearer; 200 |

### Request body

```
{
  "displayName": "  Reader Updated  "
}
```

The response has the same five profile fields, with displayName set to "Reader Updated". There is no password, role, activity flag, token hash, session data, security timestamp or internal version in this profile response.

- All query parameters and unsupported body fields are rejected. userId, _id, email, role, emailVerified, isActive, credentials and MongoDB operators cannot redirect or broaden the update.

- Successful display-name edits preserve sessions, email verification and account identity. There are no extra preferences or editable email fields yet.

## Log out the current cookie-bound session

POST /auth/logout returns 200. Send {} with mutation headers and the current refresh cookie; browsers use credentials: include. No Bearer token is required.

Response: {"success":true,"data":{"message":"Logged out."}}. Logout clears the matching cookie and revokes only that session. Other devices remain signed in. Repeated logout is safe; a network failure clears local access but cannot certify server-side revocation.

# 6. Recover a forgotten password

*Public responses avoid revealing whether an eligible account exists.*

## Request password recovery

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| POST | /auth/forgot-password | No Bearer; 202 |

### Request body

```
{
  "email": "reader@example.test"
}
```

### Same public response for known, unknown, verified or inactive addresses

```
{
  "success": true,
  "data": {
    "message": "If an eligible account exists, we will attempt to send password reset instructions."
  }
}
```

For an active eligible account, the server creates a separate hashed reset credential and attempts delivery through the configured mail adapter. A replacement invalidates older outstanding reset links. The response does not confirm inbox delivery.

## Consume the reset link

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| POST | /auth/reset-password | Reset token; 200 |

### Request body

```
{
  "token": "<reset-token-from-email>",
  "newPassword": "A replacement passphrase!",
  "passwordConfirmation": "A replacement passphrase!"
}
```

Success message: "Password reset. Please log in with your new password." Expiry defaults to 30 minutes (configurable 5-60). Unknown, expired, consumed, invalidated or unaccepted tokens are rejected; TTL cleanup is not the security check.

Reset atomically consumes the token, updates the password/passwordChangedAt, invalidates sibling reset credentials and revokes every refresh session for that user. Previously issued access tokens then fail the live user/session checks. Failed transactions roll back the complete mutation.

Links use the configured frontend /reset-password#token=... URL. The raw token is never stored in MongoDB or public responses. The screen reads it into memory and scrubs the address bar.

# 7. Change a password and verify email

*Credential changes and email ownership remain separate account actions.*

## Change a password while logged in

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| POST | /auth/change-password | Bearer; 200 |

### Request body

```
{
  "currentPassword": "A synthetic passphrase!",
  "newPassword": "A replacement passphrase!",
  "passwordConfirmation": "A replacement passphrase!"
}
```

The current password must match; reuse, weak replacements and mismatched confirmation are rejected. Success: "Password changed. Please log in again." All sessions, old access and reset links are invalidated. Email ownership is unchanged.

## Verify email ownership

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| POST | /auth/verify-email | Verification token; 200 |

### Request body

```
{
  "token": "<verification-token-from-email>"
}
```

Success: "Email verified. You can return to your account or continue reading." The transaction sets ownership, consumes the token and invalidates sibling links. It does not log in, activate the account, change its password or grant a role.

## Request a replacement verification link

| Method | Route (relative to /api) | Access / success |
|---|---|---|
| POST | /auth/resend-verification | No Bearer; 202 |

### Request body

```
{
  "email": "reader@example.test"
}
```

Same message for every valid address: "If an eligible account exists, we will attempt to send email verification instructions." Only active unverified accounts receive a new credential. Expiry defaults to 24 hours (configurable 5-1440 minutes). Use the newest accepted link; inactive accounts are not reactivated.

Trusted link: /verify-email#token=...; confirm it explicitly. Verification/reset credentials use different hash purposes and collections.

# 8. Account lifecycle and ownership

*Server-side checks determine what an account may do.*

| State or action | Current rule |
|---|---|
| New registration | Active user role; emailVerified false; attempt verification delivery; no automatic login. |
| Unverified account | Login, public reading, own profile and password management remain permitted. Features explicitly requiring ownership verification use requireVerifiedEmail. |
| Verified account | Normal operations continue; verification grants no editorial or administrative authority. |
| Inactive account | Cannot log in or perform protected operations. Old access tokens cannot bypass live inactivity/session checks. Public reading remains available. |
| Deactivation / reactivation | Internal authorized administrative process only. Atomically revoke all sessions and both credential purposes. Old sessions/links do not revive on reactivation. |
| Password reset / change | Revoke every refresh session and old access; invalidate reset links. Independent verification links retain their own validity rules. |

## Private-resource authorization foundation

The privateOwner schema plugin supplies a required immutable owner User reference. The owned-resource helper derives ownership from auth on create and scopes every individual read/update/delete to both record ID and owner ID. Foreign, missing and malformed IDs have the same safe 404 response; ownership injection/transfer is rejected.

These helpers were tested with a test-only fixture. No bookmark, progress, history, private-note, saved-passage or private-list product model/route has been implemented. The canonical editorial Note model is not private reader data.

## Privilege defaults

Only user is a defined role. Empty/unknown/reserved role checks deny access, including requests for admin. There are no public administrative endpoints or canonical write routes. Any future editorial/publishing workflow requires separately defined permissions and approval controls.

# 9. Configure the application

*Variable names and purposes only. No real secret or deployment credential is included.*

| Variable | Purpose / current constraint |
|---|---|
| PORT | Backend listening port; default 5000. |
| CONTENT_SOURCE | pdf (default) or database. PDF reads need no content DB; auth still needs MongoDB. |
| MONGODB_URI | MongoDB connection for authentication and optional database content. |
| AUTH_ENABLED | Explicit true enables auth; missing/false makes account routes unavailable. |
| AUTH_ACCESS_TOKEN_SECRET | Server-only signing secret, at least 32 bytes after trimming. |
| AUTH_ALLOWED_ORIGINS | Comma-separated exact trusted frontend origins. HTTPS in hosted deployments. |
| AUTH_TRUST_PROXY_HOPS | Controlled proxy hop count, 0-10; default 0. Do not trust arbitrary forwarding headers. |
| AUTH_COOKIE_SAME_SITE | strict, lax (default) or none. Production cookies remain Secure. |
| AUTH_ALLOW_INSECURE_LOCALHOST | Explicit development-only HTTP exception; all origins/API peer must be loopback. Forbidden in production and with SameSite=None. |
| PASSWORD_RESET_FRONTEND_URL | Trusted origin only, matching the auth allowlist; server appends /reset-password. |
| PASSWORD_RESET_TTL_MINUTES | Optional 5-60; default 30. |
| EMAIL_VERIFICATION_FRONTEND_URL | Trusted origin only, matching the auth allowlist; server appends /verify-email. |
| EMAIL_VERIFICATION_TTL_MINUTES | Optional 5-1440; default 1440. |
| MAIL_PROVIDER | resend, selected explicitly. |
| MAIL_DELIVERY_ENABLED | Explicit true opt-in; otherwise no mail is sent. |
| MAIL_FROM | Provider-verified sender. |
| RESEND_API_KEY | Server-only sending credential. |
| VITE_API_URL | Frontend public backend origin, e.g. http://localhost:5000, without /api. Never place secrets in VITE variables. |

The last implementation check found mail credentials, sender/provider and verification origin unconfigured, with delivery disabled. Enabling mail requires real provider/domain setup and a delivery check; this note does not claim live delivery.

# 10. Initialize and integrate

*Setup actions are explicit; this note does not execute them against your configured database.*

## Backend setup

### Run only against the intended development/deployment environment

```
cd backend
npm install
# Configure environment through your own secure setup.
npm run auth:init
npm run dev
```

auth:init creates users, refreshsessions, passwordresettokens and emailverificationtokens with their declared indexes. It seeds no user, drops no indexes and touches no manuscript collection. Review existing conflicting indexes/duplicates separately. This note did not run it against Atlas.

Reset, verification and lifecycle mutations require a transaction-capable replica set/mongos deployment. Standalone MongoDB keeps core login but these mutations fail closed. Missing token indexes also fail closed. Routine profile editing adds no new collection or migration.

## Frontend routes

| Path | Purpose |
|---|---|
| /login | Log in, then navigate to account settings. |
| /account/settings | View profile, edit display name, see verification, change password or log out. |
| /account/change-password | Authenticated current/new-password form. |
| /forgot-password | Account-neutral recovery request. |
| /reset-password | Read the recovery fragment into memory, scrub it and submit a reset. |
| /verification-pending | Explain pending ownership and request another verification link. |
| /verify-email | Confirm the verification link, then navigate to login or reading. |

Set VITE_API_URL before building the frontend, keep account requests in credentials: include mode, and retain the shared auth service rather than exposing its access token. SPA rewrites must support direct route visits.

## Typical account flow

Register -> receive/confirm verification when delivery is enabled -> log in -> GET /users/me -> PATCH /users/me -> optionally change password or log out. Reading can happen before, during or without this account flow.

# 11. Errors and abuse limits

*Errors use a safe public message rather than driver details or submitted secrets.*

### Typical protected-route error (401)

```
{
  "success": false,
  "message": "Authentication required."
}
```

| Status | Meaning in the current API |
|---|---|
| 400 | Malformed JSON, bad account/resource input, unsupported fields, invalid/expired/used recovery or verification link. |
| 401 | Missing/invalid/expired authentication, revoked session, inactive/missing user, stale password claim; login uses a neutral credential error. |
| 403 | Bad Origin/CSRF protection, a required verified-email check, or a denied role operation. |
| 404 | Unknown route/content; foreign or unknown private-resource ID. No general /users/:id account endpoint. |
| 409 | Duplicate normalized registration email. |
| 413 / 415 | JSON body exceeds 1 MB / account mutation is not application/json. |
| 429 | Abuse limit reached; Retry-After is supplied for an exceeded bucket. |
| 503 | Authentication/recovery/verification configuration, indexes or required storage are unavailable. |
| 500 | Sanitized unexpected failure. Database-provider search is not implemented. |

## Current limits - 15-minute windows

| Operation | Limit |
|---|---|
| Register / login / logout | 20 / 10 / 60 requests per IP |
| Forgot-password / resend-verification | 5 per IP and 3 per normalized-email hash |
| Reset-password / verify-email | 10 per IP |
| Change-password | 5 per IP |
| Profile PATCH | 30 per IP |

Limiters are bounded but in-process, reset at restart and are not shared across instances. A multi-instance deployment needs a shared/edge limiter. Generic forgot/resend responses do not prove cryptographically constant-time database behavior.

# 12. Readiness, limits and source notes

*Separate implemented behavior from live deployment verification and follow-up work.*

## Implemented and recorded validation

| Last implementation record (Task 7) | Result |
|---|---|
| Backend tests | 158 passed on the final full rerun; includes 15 new profile checks. |
| Frontend tests | 27 passed, including 5 new profile checks. |
| Frontend lint / production build | Passed; the existing large-bundle warning remains. |
| PDF/source consistency guard | 32 checks passed; 10 data files unchanged; databaseModified false; importApproved false. |

These are the recorded Task 7 results, not a fresh test run performed while making this PDF. An initial concurrent backend run had one unreproduced failure; an unchanged-code backend-only rerun passed all 158. Its initial cause was not confirmed.

## What remains unfinished

- Live provider delivery and deployment browser/cookie behavior are unverified. The in-app browser connection was unavailable.

- Email delivery uses a bounded, best-effort in-memory queue shared by reset/verification. Crashes can lose pending work; provider acceptance is not inbox delivery. There are no durable delivery guarantees or automatic retries.

- Email-address change, automatic refresh/rotation, logout-all, session listing, public administration and reader personal-data features are not implemented.

- Default PDF content availability does not approve or publish a database edition. Database content retains its separate approval/publication filters. No canonical write endpoints exist.

## Where the contract comes from

Routes/controllers: backend/src/app.js; routes/book.routes.js, auth.routes.js and user.routes.js; controllers/book.controller.js, auth.controller.js, passwordRecovery.controller.js, emailVerification.controller.js and user.controller.js.

Behavior and configuration: shared/manuscript.mjs and manuscript.json; backend/src/services/readContent.service.js, auth.service.js, passwordRecovery.service.js, emailVerification.service.js and user.service.js; config/auth.js; middleware/auth.middleware.js; models/User.js.

Detailed handoffs: backend/docs/AUTHENTICATION.md, PASSWORD_RECOVERY.md, EMAIL_VERIFICATION.md, AUTHORIZATION.md and USER_PROFILES.md. The supplied Timely PDF influenced organization and layout only; no Timely route, credential or product rule is imported.